

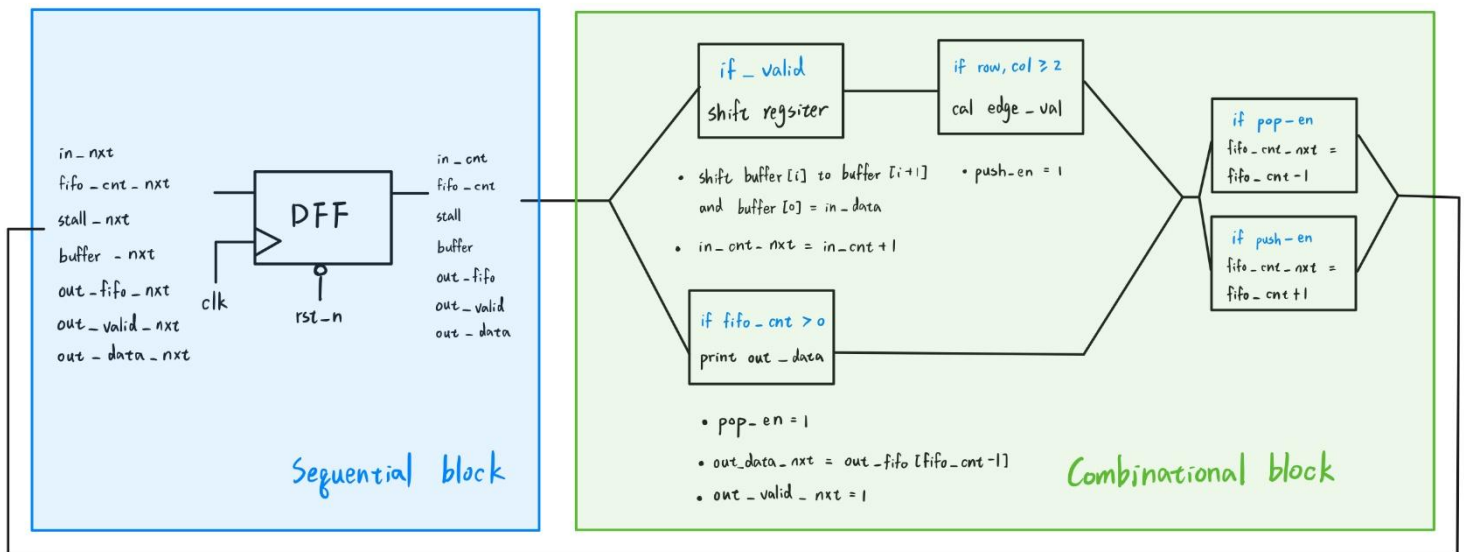
HW02 Report

Name: 陳欣怡

Student ID: 113511116

Server Account: dcs093

Architecture



Design Methodology

● 整體架構：

將變數的 current state 跟 next state 分開，next state 在 combinational block 計算完後，在下一個 clk 上升緣再賦值進 current state。

1. 在 `in_valid` 時將 `in_data` 存到 `buffer`，並記錄已經存到第幾個輸入。
2. 當 `buffer` 存到第三行的第三個數後就可以開始計算 output pixel，並將計算結果存入 `out_fifo`，且將 `push_en` 設為 1。後每輸入一次 `in_data` 就會計算一次直到輸入結束。
3. 若非 `in_valid` 時則可以將已計算好的結果輸出，將 `pop_en` 設為 1。
4. 在 combinational block 的最後需要判斷是否有 `pop_en` 或 `push_en` 發生，若 `pop` 要將 `fifo_cnt-1`，若 `push` 則將 `fifo_cnt+1`，`fifo_cnt` 代表有幾個計算好的結果等待輸出。

● 面積優化：

1. **改變輸入資料的儲存方式：**一開始我設了三個[0:7]的陣列作為輸入的暫存器，當第三行結束時再將後兩行賦值到前兩行，第三行則繼續存取新的數值然後計算。後來發現計算一個 `out_data` 實際只需要現在輸入的 `in_data` 再加上前 18 個數(如下圖)就好，因此我把原本 3*7 個 8bit 的陣列改成 1*18 個 8bit 的 `buffer`，面積得到大幅的下降。

```

/*
buffer[17], buffer[16], buffer[15]  buffer[14], buffer[13], buffer[12], buffer[11], buffer[10],
buffer[9],  buffer[8],  buffer[7],  buffer[6], buffer[5],  buffer[4],  buffer[3],  buffer[2],
buffer[1],  buffer[0],  in_data
*/
if ((row_cnt >= 2) && (col_cnt >= 2)) begin
    push_en = 1'b1;
    edge_val = cal_out(
        buffer[17], buffer[16], buffer[15],
        buffer[9],  buffer[8],  buffer[7],
        buffer[1],  buffer[0],  in_data
    );
    out_fifo_nxt = {out_fifo[34:0], edge_val};
end

```

2. **改變計算結果的取出方式**：我原先在 out_fifo 設了兩個指標，來記錄寫入到底幾個與讀出到第幾個，然而這樣每次在寫入與讀出時都會有一個 36 的 MUX。後來改成動態寫入與移位讀出的方式(out_fifo_nxt[fifo_cnt] = edge_val;，固定讀出 out_fifo[0]再將 out_fifo 往右 shift 擠掉舊的值)面積有變少一點，然而面積真正大幅下降是改成移位寫入與動態讀出 `out_fifo_nxt = {out_fifo[34:0], edge_val};` `out_data_nxt = out_fifo[fifo_cnt-1];`，這個寫法會比較小是因為動態寫入的 out_fifo_nxt[fifo_cnt] = edge_val;會在 out_fifo 的 36 個 DFF 前每一個都需要 MUX 來判斷是要保持舊值還是存入新資料，而移位寫入動態讀出則只需要一個 MUX 來決定哪個資料要被讀出，因此面積較小。

Reflection

覺得這次作業非常有趣，從慢慢理解題意到用最笨的方法寫出來，之後再慢慢優化，想新的架構、算法，每個步驟都讓我獲得滿滿的成就感。相比 HW01 在寫這個作業的時候我對於語法也比較熟悉，至少能將自己的想法轉為程式碼，而不至於無從下手。但是不同於 HW01 的純邏輯設計，這次作業多了 sequential 的概念，也是我覺得最困難的地方，一開始我把太多邏輯運算寫在 sequential block 裡，導致我在 debug 時非常難想，後面將變數分成兩個狀態，統一在 combinational block 做運算再賦值，不只程式碼變乾淨了，在 coding 時也比較不易出錯！

AI-use Log

- What tools/models you used? (e.g., Gemini, ChatGPT, Copilot, ...)
GPT-5.3, Gemini 3.1 pro
- Key Prompts: 3–5 primary prompts (paste the original text or conversation link)
 - (貼程式碼) 這段是否有語法錯誤，應該怎麼寫比較好
 - (貼程式碼) 確保結果正確下如何優化面積
- What and why you adopt or reject? How you verify generated output?
若看得懂 AI 給的改動建議，並且認為有達到優化的效果，我就會把它放到程式碼裡然後 Demo 看看是否有達到預期效果。